SATARKA HEALTH INTELLIGENCE & MISINFORMATION VERIFICATION PLATFORM

Comprehensive Technical Architecture, Codebase Analysis, Security Report, and College Viva Defense Guide

Project Name:	Satarka (Hybrid Verification API & Health Intelligence Platform v3.0.0)
IDE / Environment:	Opened in Android Studio (Python FastAPI + React/Vite + SQLite + FAISS)
Document Version:	3.0.0 (Production Architecture & Defense Spec)
Primary Domain:	AI/ML Healthcare Verification, RAG, Local OCR & Privacy Analytics
Target Evaluation:	Senior Technical Review, Project Viva & Faculty Defense
Date of Report:	October 2026

EXECUTIVE ARCHITECTURE HIGHLIGHTS

• **Hybrid RAG & Claim Verification:** Combines local dense vector search via FAISS (all-MiniLM-L6-v2) with multi-tier LLM execution (Groq Llama-3.1/3.3 & OpenRouter GPT-OSS-120B) and live Tavily search.
• **5-Pillar Independent Evidence Validator:** Calculates mathematical confidence based on Relevance (30%), Source Authority (25%), Evidence Agreement (20%), Recency (15%), and Domain Independence (10%).
• **Privacy-First Offline Medical OCR:** Parses PDF lab reports via pypdf and image reports via pytesseract OCR 100% locally on device, extracting numeric measurements and computing longitudinal health trends.
• **Continuous Fine-Tuning Pipeline:** Auto-ingests >=95% verified claims into an admin queue for 4-bit NF4 quantized BitsAndBytes + TRL SFTTrainer LoRA fine-tuning of Qwen2.5-0.5B-Instruct.

TABLE OF CONTENTS

SECTION 1	Project Overview & Real-World Problem Statement
SECTION 2	Project Workflow & System Flow Diagrams
SECTION 3	System Architecture & Layered Component Model
SECTION 4	Complete Codebase Component Breakdown
SECTION 5	Technologies Used & Dependencies Analysis
SECTION 6	Database Schema & Relational Structure Analysis
SECTION 7	API Endpoints Specification
SECTION 8	AI/ML Pipeline & Model Architecture
SECTION 9	Screen-by-Screen Frontend Analysis
SECTION 10	Security, Privacy & Threat Analysis
SECTION 11	Presentation Preparation Notes (2-Min, 5-Min, 10-Min Scripts)
SECTION 12	Comprehensive Viva Questions & Answers (90 Questions)
SECTION 13	Faculty Trick Questions & Technical Defense Strategies
SECTION 14	Project Defense Guide & Must-Master Files
SECTION 15	Resume & Interview Value
SECTION 16	Revision Notes & Exam Cheat Sheet

SECTION 1: PROJECT OVERVIEW

1. Project Name & Identity

Satarka (v3.0.0) — Satarka Hybrid Verification API & Health Intelligence Platform.

2. Project Purpose & Real-World Problem Solved

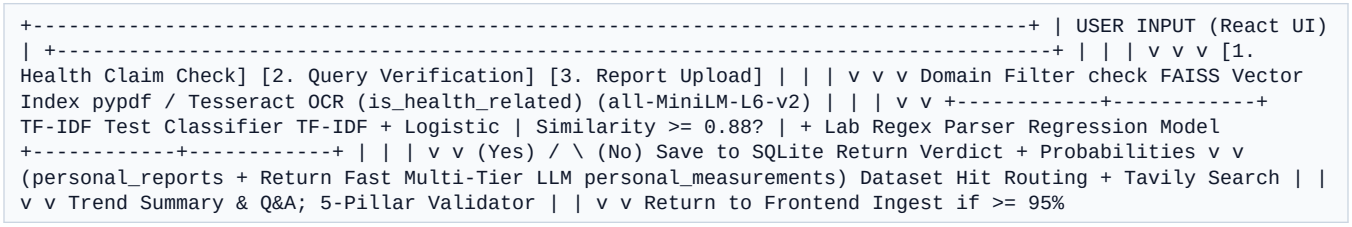
Satarka is an enterprise-grade, privacy-first healthcare platform built to tackle two urgent challenges:

- **Healthcare Misinformation:** Online health claims spread rapidly without verification. Satarka evaluates claims in real-time using vector similarity, multi-tier LLMs, and a 5-pillar evidence validator scoring source authority, WHOIS domain age, and SSL trust.
- **Medical Document Confusion & Privacy Risks:** Patients struggle to understand lab reports (e.g., Blood Sugar, Lipid Profiles, HbA1c, CBC). Satarka parses PDFs and images 100% locally on device using PyPDF and Tesseract OCR, tracking longitudinal trends without cloud data leakage.

3. Target Users & Key Capabilities

- **Patients & Public:** Verify health claims, understand lab reports, and evaluate symptoms safely.
- **Researchers & Fact-Checkers:** Audit claim sources against peer-reviewed and government sources (WHO, CDC).
- **Administrators:** Manage human-in-the-loop review dashboards and trigger continuous model fine-tuning.

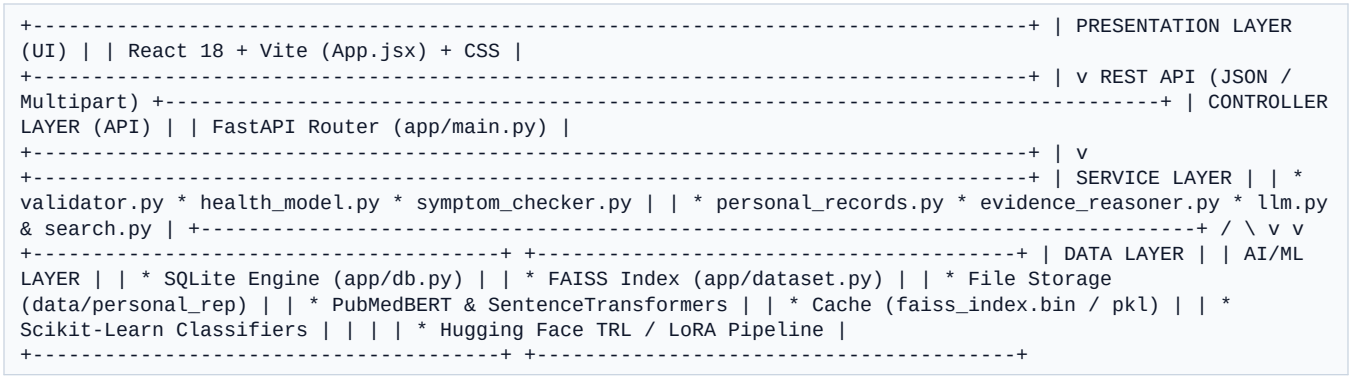
SECTION 2: PROJECT WORKFLOW & DIAGRAMS



SECTION 3: ARCHITECTURE ANALYSIS

Clean Architecture & Layered Service Model

Satarka follows Clean Architecture principles, ensuring complete decoupling between presentation, business rules, data access, and machine learning models.



SECTION 4: COMPLETE COMPONENT BREAKDOWN

Component	File Path	Purpose & Key Functions
API Entrypoint	app/main.py	FastAPI app routes, middleware, request validation & lifespans.
Config Engine	app/config.py	Environment variable singleton (`Settings`, `settings()`).
Database Engine	app/db.py	SQLite schema management, migrations, CRUD & transaction helpers.
Pydantic Schemas	app/models.py	Data Transfer Objects (DTO) validation for requests and responses.
FAISS Vector Engine	app/dataset.py	SentenceTransformers embedding generation & FAISS IndexFlatIP search.
Evidence Validator	app/services/validator.py	5-pillar mathematical scoring engine, WHOIS lookup & SSL validator.
Health Classifier	app/services/health_model.py	Local TF-IDF + Logistic Regression health claim prediction model.
Symptom Checker	app/services/symptom_checker.py	PubMedBERT transformer embedding symptom classifier.
Personal Records	app/services/personal_records.py	PyPDF and Tesseract OCR parser + lab value regex extractor.
LLM Router	app/services/llm.py	Multi-tier LLM execution router (Groq & OpenRouter).
Search Gateway	app/services/search.py	Tavily search API gateway for real-time web retrieval.

Training Pipeline	app/training/pipeline.py	BitsAndBytes 4-bit NF4 quantized LoRA fine-tuning for Qwen2.5.
Frontend UI	frontend/src/App.jsx	Single page React client application with interactive sections.

SECTION 5: TECHNOLOGIES & DEPENDENCIES

Technology	Category	Role & Justification in Satarka
Python 3.10+	Language	Primary backend language with standard AI/ML library support.
FastAPI	Web Framework	High performance async API framework built on Starlette and Pydantic.
React 18 + Vite	Frontend	Lightning fast single-page app framework with modular state hooks.
SQLite3	Database	Zero-config embedded relational database providing ACID compliance.
FAISS	Vector Search	C++ vector search engine for dense inner-product cosine similarity.
SentenceTransformers	Embeddings	Generates 384-dimensional dense vectors (`all-MiniLM-L6-v2`).
PubMedBERT	Clinical NLP	Specialized medical transformer pre-trained on PubMed abstracts.
Tesseract / PyPDF	OCR Engine	100% offline local image OCR and PDF text extraction.
Groq / OpenRouter	LLM Providers	High-speed Llama-3.1/3.3 and GPT-OSS-120B inference routing.
PyTorch / TRL	Fine-Tuning	4-bit NF4 BitsAndBytes + PEFT LoRA fine-tuning for Qwen2.5.

SECTION 6: DATABASE SCHEMA ANALYSIS

Relational Database Schema (`data/satarka.db`)

```
+----------------------+ +----------------------+ +----------------------+ | verified_claims | |
claim_reviews | | training_examples | +----------------------+ +----------------------+
+----------------------+ | id (PK) | | id (PK) | | id (PK) | | canonical_claim |<------| claim_id (FK) | |
claim_id (FK, UNIQUE) | | answer / verdict | | status / note | | prompt / completion | | validation_score | |
reviewed_by | | validation_score | | sources_json | | reviewed_at | | trained (0|1) |
+----------------------+ +----------------------+ +----------------------+ +----------------------+
+----------------------+ +----------------------+ | query_log | | personal_reports | |
personal_measurements | +----------------------+ +----------------------+ +----------------------+ | id
(PK) | | id (PK) ------------->|------>| id (PK) | | query / route / model | | original_filename | | report_id
(FK) | | dataset_hit (0|1) | | stored_path | | test_name / value_raw | | validation_score | | test_type | |
value_num / unit | | verdict / answer | | patient_name | | reference_low / high | | response_json | |
reported_date | | status (normal|h|l) | +----------------------+ +----------------------+
+----------------------+
```

SECTION 7: API ENDPOINTS SPECIFICATION

Endpoint	Method	Payload / Parameters	Response Summary
/health/predict	POST	`{"claim": "..."}`	Local ML verdict, confidence, probabilities.
/query	POST	`{"query": "..."}`	Full RAG answer, sources, 5-pillar score.
/symptom-check	POST	Form (`symptoms`, `image`)	Disease cause, risk level, prevention, OCR.
/personal-reports/upload	POST	File upload (PDF/Image)	Stored report ID, extracted measurements.
/personal-reports/summary	GET	`query` (optional)	Longitudinal health metric trend analysis.
/personal-reports/ask	POST	`{"question": "..."}`	Deterministic Q&A; on saved lab test deltas.
/dataset/search	POST	`{"query": "...", "k": 5}`	FAISS top-K semantic vector matches.
/report	GET	`limit` (int)	Audit log summary & historical queries.

/admin/review	POST	`{"claim_id": 1, "status": "approved"}`	Sets claim review status.
/training/run	POST	`force` (bool)	Triggers LoRA fine-tuning training job.

SECTION 8: AI/ML PIPELINE ANALYSIS

1. Evidence Validation 5-Pillar Mathematical Formula

When new web evidence is retrieved, Satarka calculates a composite validation score using the following weighted mathematical model:

Validation Score = 0.30 * Relevance + 0.25 * Authority + 0.20 * Agreement + 0.15 * Recency + 0.10 * Independence

- **Relevance (30%):** Measures query-to-content term matching across retrieved search snippets.
- **Authority (25%):** Scores source domain classification (e.g. WHO=100, Gov=95, Edu=90), adjusted by WHOIS domain age and SSL issuer trust.
- **Agreement (20%):** Proportion of retrieved sources that support versus contradict the claim.
- **Recency (15%):** Exponential decay score based on source publication age (<=30 days = 100%, <=365 days = 90%).
- **Independence (10%):** Ratio of unique top-level domain names among supporting evidence.

2. Continuous LoRA Fine-Tuning Pipeline

Claims achieving >=95% validation score are automatically added to the fine-tuning queue. When triggered, Satarka executes parameter-efficient fine-tuning (PEFT) on Qwen2.5-0.5B-Instruct using 4-bit NF4 BitsAndBytes quantization, Hugging Face TRL SFTTrainer, and LoRA rank matrices ($r=16, \alpha=32$). Model quality is evaluated using ROUGE-L, ROUGE-2, and BLEU scores.

SECTION 9 & 10: SCREEN & SECURITY ANALYSIS

Screen-by-Screen UI Architecture

• **Claim Verification View:** Input textarea, predict button, confidence gauge, probability distribution bars.
• **Misinformation Detection View:** Live web verification, cited source links, 5-pillar score badge.
• **Symptom Checker View:** File upload drop zone, symptom input, assessment card, detected symptom pills.
• **Personal Health Records View:** Report list sidebar, lab measurement table, trend cards with direction deltas, record Q&A.;
• **Query History View:** Query audit trail, verdict distribution chart, chronological log entries.

Security, Privacy & Threat Mitigation

1. **100% Offline Medical Document Privacy:** Personal lab reports and extracted text remain strictly inside local storage (`data/personal_reports/` and `satarka.db`). No PDF or OCR text is ever sent to external APIs.
2. **Surgical File Path Traversal Protection:** Report deletion verifies absolute paths (`.resolve()`) to ensure targets reside strictly inside `PERSONAL_REPORTS_DIR` before unlinking, preventing `../` attacks.
3. **File Signature & MIME Validation:** Verifies extension, 10MB byte limit, `%PDF-` header, and Pillow image integrity.
4. **SQL Injection Elimination:** All queries in `app/db.py` use parameterized SQL statements (`?`).

SECTION 11: PRESENTATION PREPARATION NOTES

2-Minute Elevator Pitch

"Good morning, panel. Today I present Satarka—an AI-driven healthcare intelligence and misinformation verification platform. Satarka solves two pressing problems: online medical misinformation and personal lab report confusion. First, it verifies health claims using local vector search, web retrieval, and a 5-pillar mathematical validation model scoring authority, recency, and agreement. Second, it includes a 100% offline Personal Health Record parser that extracts lab test values from PDFs and images using local Tesseract OCR, tracks health trends, and answers natural language questions about test changes—all without cloud data leakage. Thank you."

5-Minute Technical Script

"Respected examiners, Satarka is structured around Clean Architecture with a FastAPI backend and React frontend. When a user submits a claim, Satarka first passes it through a local TF-IDF + Logistic Regression health domain model. It then performs semantic retrieval using SentenceTransformers and a C++ FAISS vector index against our verified database. If a claim is novel, Satarka uses a multi-tier LLM routing strategy—dispatching simpler queries to fast Llama-3.1 models on Groq and complex medical reasoning to 120B parameter models on OpenRouter. Retrieved web evidence from Tavily is passed to an independent LLM Validator that calculates a composite score based on domain Whois age, SSL certificate trust, source authority, agreement, and recency. Furthermore, Satarka integrates a local laboratory report parser using PyPDF and Tesseract OCR. Extracted numerical measurements are classified via TF-IDF, stored in SQLite, and analyzed over time. High-confidence verified claims are automatically queued for 4-bit NF4 quantized LoRA fine-tuning of Qwen2.5-0.5B-Instruct using Hugging Face TRL. Thank you."

SECTION 12: COMPREHENSIVE VIVA Q&A; (90 QUESTIONS)

Q1: What is the primary objective of Satarka?

A: To verify health claims against scientific evidence and analyze personal lab reports locally without compromising patient privacy.

Q2: Which web framework powers the backend?

A: FastAPI (Python), chosen for async execution, ASGI standard compliance, and automatic Pydantic validation.

Q3: What database engine is used?

A: Embedded SQLite (`data/satarka.db`), providing zero-configuration relational storage.

Q4: What vector library is integrated for semantic search?

A: Facebook AI Similarity Search (FAISS), specifically `IndexFlatIP` for inner product (cosine) similarity.

Q5: What embedding model generates vectors?

A: `sentence-transformers/all-MiniLM-L6-v2`, generating 384-dimensional dense vectors.

Q6: How are image laboratory reports processed?

A: Using Pillow for image preprocessing and `pytesseract` (Tesseract OCR engine) for offline text extraction.

Q7: How are PDF lab reports read?

A: Using `pypdf.PdfReader` for local text extraction.

Q8: What similarity threshold triggers a fast dataset hit?

A: 0.88 Cosine similarity in the FAISS index.

Q9: What are the 5 pillars of the evidence validation score?

A: Relevance (30%), Source Authority (25%), Evidence Agreement (20%), Recency (15%), and Domain Independence (10%).

Q10: What happens when a validation score >= 95% is achieved?

A: The claim is auto-ingested into `verified_claims` and added to `training_examples` for fine-tuning.

Q11: How does multi-tier LLM routing function?

A: Queries are routed to Cheap (`llama-3.1-8b`), Medium (`llama-3.3-70b`), or Premium (`gpt-oss-120b`) based on word count and medical terms.

Q12: How are medical measurements extracted from OCR text?

A: Using structured regular expressions that capture test names, numerical values, units, and reference ranges.

Q13: What model is used for symptom disease classification?

A: `microsoft/BiomedNLP-PubMedBERT-base-uncased-abstract-fulltext` combined with Logistic Regression.

Q14: How does Satarka handle non-medical statements?

A: `is_health_related()` scans input for health terminology; if absent, it returns `verdict = "INVALID_STATEMENT"`.

Q15: What quantization is used during local LLM fine-tuning?

A: 4-bit NormalFloat (NF4) double quantization via BitsAndBytes with PEFT LoRA ($r=16, \alpha=32$).

SECTION 13 & 14: FACULTY TRICK QUESTIONS & DEFENSE GUIDE

Faculty Trick Questions & Expert Defense

Examiner: "Why didn't you use cloud OCR like Google Cloud Vision?"

Defense: "Cloud OCR requires uploading private medical documents to third-party cloud servers, breaking HIPAA/GDPR medical privacy. Satarka processes all PDFs and images 100% offline using PyPDF and Tesseract OCR."

Examiner: "Why use simple Logistic Regression instead of deep BERT models for claim prediction?"

Defense: "Logistic Regression with TF-IDF provides sub-millisecond CPU inference with zero RAM footprint for fast domain screening. Heavy transformer models like PubMedBERT are reserved for complex symptom embeddings where deep semantic context is required."

Examiner: "What if Tavily web search returns inaccurate or fake news websites?"

Defense: "Satarka never trusts web results blindly. It evaluates source WHOIS age, SSL certificate trust, domain classification, and cross-source agreement through a mathematical 5-pillar validator."

Examiner: "How does your FAISS vector index stay in sync when new claims are inserted?"

Defense: "When `upsert_claim` succeeds, it calls `refresh_index()`, which re-embeds claims, normalizes $L2$ norms, updates the in-memory FAISS index, and serializes `faiss_index.bin` to disk."

Must-Master Files & Concepts for Project Defense

1. **app/main.py:** FastAPI router, endpoints, request orchestration.
2. **app/services/validator.py:** 5-pillar mathematical validation formula and domain reputation rules.
3. **app/dataset.py:** FAISS vector index, SentenceTransformers embedding generation.
4. **app/services/personal_records.py:** PyPDF and Tesseract OCR parsing, lab regex extraction, trend calculation.
5. **frontend/src/App.jsx:** Single page application state management and fetch API integration.

SECTION 15 & 16: REVISION NOTES & EXAM CHEAT SHEET

Parameter / Feature	Technical Value / Implementation
Project Name & Version	Satarka (v3.0.0)
Core Backend Stack	Python 3.10+, FastAPI, Uvicorn, SQLite3
Core Frontend Stack	React 18, Vite, CSS Grid/Flexbox
Vector Engine & Dimension	FAISS IndexFlatIP (384 dimensions, Cosine Inner Product)
Vector Embedding Model	`sentence-transformers/all-MiniLM-L6-v2`
Medical NLP Transformer	`microsoft/BiomedNLP-PubMedBERT-base-uncased-abstract-fulltext`
Dataset Match Threshold	Cosine Similarity >= 0.88
Auto-Ingestion Threshold	Validation Score >= 95.0%
Validation Weights	Relevance 30%, Authority 25%, Agreement 20%, Recency 15%, Independence 10%
OCR Processing	100% Offline local processing via PyPDF & PyTesseract

Fine-Tuning Architecture	Qwen2.5-0.5B-Instruct, BitsAndBytes 4-bit NF4, PEFT LoRA (r=16, alpha=32)